

Object-Oriented Abstraction in Java and C++

A Focused Comparative Design Study

T. R. Bentley

Repository: Tybent18/oop-concepts
Document class: repository-grounded technical report
Edition: September 2026

Scope

Encapsulation, inheritance, polymorphism, and overriding in Java and C++.

Evidence statement

Claims in this report are limited to repository artifacts identified by filename. Proposed tests, benchmarks, integrations, and research directions are labeled as future work; they are not reported results.

Abstract

This concise design report studies two focused object-oriented examples: a Java shape hierarchy and a C++ animal-inheritance exercise. The analysis centers on shared abstraction, specialization, overriding, and polymorphic dispatch. Because the evidence base contains only two examples, the document is deliberately scoped as a comparative technical note rather than a broad claim about object-oriented architecture.

1. Repository evidence map

Cluster	Artifacts	Interpretation
Shape hierarchy	shape_area_calculator.java	An abstract shape contract with specialized area implementations.
Animal hierarchy	animal_inheritance.cpp	A C++ inheritance example demonstrating specialized behavior.

2. Technical framing

- Which behavior belongs in a base abstraction?
- When does overriding reduce duplication without hiding intent?
- How do Java and C++ express related object models differently?

3. Reproducible inspection protocol

A reviewer should clone the repository, record the commit SHA and toolchain versions, inspect each mapped artifact, execute only examples whose entry points and prerequisites are declared, preserve standard output and errors, and compare observations with documented expectations. Failures in deliberate debugging targets are evidence only when the expected failure class is stated in advance.

4. Evidence maturity

Level	Meaning	Current interpretation
E0	Artifact listed	Source file exists in the repository
E1	Behavior described	README or report states intended behavior
E2	Execution recorded	Command, environment, and output are retained
E3	Behavior tested	Repeatable assertions evaluate expected behavior
E4	Comparative result	Frozen data supports a bounded comparison

5. Limitations

- Two examples cannot support general performance or maintainability conclusions.
- No design-pattern catalog or production framework is implemented.
- The Java filename differs from its public Program class and requires a temporary rename to compile conventionally.

6. Development roadmap

- Add tests for subtype behavior and invalid inputs.
- Introduce composition as a comparison to inheritance.
- Add class diagrams generated from the implemented code.

7. Portfolio role

This repository belongs to a cumulative foundation sequence: Foundations & Algorithms → Data Structures Practice → OOP Concepts → Math for Computing → Practical Utilities. The sequence demonstrates increasing abstraction and application; advanced repositories carry the stronger systems and empirical-research claims.

Conclusion

The repository is most credible when its modest scale is treated as a strength: each artifact can be inspected directly, every claim can point to code, and future measurements can be added without rewriting history. Sophistication comes from traceability, not inflated labels.